

JARVIS

Software Requirements Document (SRD)

Retail Stock Trading Application

Prepared by: Zeeshan Khateeb

Prepared for: Jarvis Consulting Group

Client / Stakeholder: Trevor Doto

Contents

1. Introduction	3
1.1 Executive Summary	3
1.2 Purpose.....	3
1.3 Intended Audience	3
1.4 Scope.....	4
2. Overall Description.....	4
2.1 Business Context	4
2.2 User Needs	4
2.3 Assumptions and Dependencies	5
3. System Overview	5
4. User Stories.....	5
Authentication and Account Access	5
Trading	5
Portfolio Monitoring.....	6
Notifications	6
5. Stakeholders	6
6. Use Case Diagram	6
7. System Architecture	7
8. Trading Workflow	9
9. Data Model (ER Diagram).....	10
10. Functional Requirements	12
11. Data Requirements	13
12. Non-Functional Requirements	14
13. Infrastructure and Monitoring	15
14. Future Enhancements	16

1. Introduction

1.1 Executive Summary

The Retail Banking Stock Trading Application is a digital trading platform designed to allow retail banking customers to invest directly in stock markets without relying on traditional broker- assisted trading. Currently, customers of the bank must schedule appointments or communicate with a broker through phone calls or physical visits to execute stock trades. This process introduces friction and slows down investment decisions.

The proposed application aims to provide customers with a modern, easy-to-use mobile and web interface similar to platforms such as Robinhood or Questrade. Through this application, users will be able to monitor stock prices, place buy or sell orders, and track the performance of their investments directly from their banking ecosystem. The system will prioritize simplicity and accessibility so that even non-technical users can easily check their investments and perform trades.

The Minimum Viable Product (MVP) will initially support buying and selling stocks listed on North American exchanges. More advanced trading instruments such as options trading, leveraged trading, foreign exchange markets, and cryptocurrency will not be supported in the initial release but the system will be designed in a way that allows future expansion to support these features.

1.2 Purpose

The purpose of this Software Requirements Document (SRD) is to define the functional and non-functional requirements for the Retail Banking Stock Trading Application. The document serves as a communication tool between stakeholders, business analysts, software developers, and system architects to ensure alignment on the scope and expectations of the system.

It outlines system capabilities, constraints, assumptions, and performance expectations gathered during the elicitation meeting with the project stakeholder. The SRD will guide system design, development, testing, and deployment activities throughout the project lifecycle.

1.3 Intended Audience

This document is intended for several stakeholders involved in the development and operation of the system. These include business stakeholders who define the business goals of the project, software development teams responsible for implementing the system, quality assurance teams who validate system behavior, and operations teams responsible for maintaining the application infrastructure. Security and compliance officers will also reference this document to ensure that financial and privacy regulations are satisfied.

1.4 Scope

The system will provide retail bank customers with the ability to participate in stock trading directly through a secure application connected to their existing bank accounts. The platform will allow users to view stock information, place trades, monitor the performance of their investments, and receive notifications related to their trading activity.

The application will integrate with the bank's existing customer accounts so that authentication and account management can leverage existing banking infrastructure. While the platform will enable trading activity using funds from linked bank accounts, direct deposits and withdrawals will continue to be managed through the bank's existing financial systems rather than through the trading application itself.

The MVP will primarily focus on core trading functionality including account authentication, stock search, order placement, portfolio monitoring, and notifications.

2. Overall Description

2.1 Business Context

The Retail application currently provides brokerage services that require customers to interact directly with stock brokers to execute trades. This traditional approach limits customer participation and slows the trading process. The new trading platform aims to eliminate these barriers by allowing customers to independently access financial markets through a digital interface.

The platform is designed to reduce friction for users while keeping them within the bank's ecosystem. This strategy prevents customers from moving their investment activities to third-party trading platforms and helps the bank expand its financial services offering.

During the elicitation meeting, the stakeholder emphasized that the application should be intuitive enough that even less technically experienced users, such as older customers, can easily log in, check stock prices, and execute trades with minimal effort.

2.2 User Needs

Retail investors require a system that allows them to manage their investments independently while maintaining strong security standards. Users must be able to log into the application quickly, monitor stock prices, and execute trades without needing to communicate with a broker.

Additionally, investors require the ability to view historical transactions, monitor the value of their portfolios, and receive notifications when important changes occur in the market or in their investment accounts.

2.3 Assumptions and Dependencies

The application will rely on existing bank infrastructure for user authentication and personal data management. Instead of storing personal banking information directly within the trading platform, the system will reference existing bank accounts through unique identifiers such as bank account numbers.

Stock price data will be retrieved through external APIs that provide market information. Due to the cost associated with real-time market data feeds, the application will initially use delayed stock market data.

The system will also depend on secure cloud infrastructure to support scalability and reliability.

3. System Overview

The system will consist of several primary components working together to deliver trading functionality to end users.

Users will access the system through a mobile or web application interface. The application will authenticate users using existing bank credentials and optional biometric authentication mechanisms such as fingerprint or facial recognition. Once authenticated, the system will allow users to search for stocks, view price information, and execute buy or sell orders.

Trade execution requests will be transmitted to backend services responsible for processing orders and storing transaction records. The system will maintain a record of all trades, including the date, quantity of shares purchased, and purchase price.

Additionally, the system will retrieve stock price data from external financial data providers. This information will be displayed to users through the application interface and will also be used to calculate the real-time valuation of each user's portfolio.

4. User Stories

The following user stories describe the key system behaviors from the perspective of the end user.

Authentication and Account Access

As a retail customer, I want to log into the trading application using my bank account credentials so that I can securely access my investment account. The system should support additional authentication methods such as biometric verification or multi-factor authentication to enhance security.

Trading

As a user, I want to search for stocks and view their current price and historical performance so that I can make informed investment decisions. The application must allow users to

submit buy or sell orders for stocks listed on supported exchanges.

Portfolio Monitoring

As an investor, I want to view my portfolio performance and track the value of my investments over time. The system should calculate gains and losses based on trade history and current market prices.

Notifications

As a user, I want to receive alerts when significant changes occur in stocks that I own or monitor so that I can react quickly to market changes.

5. Stakeholders

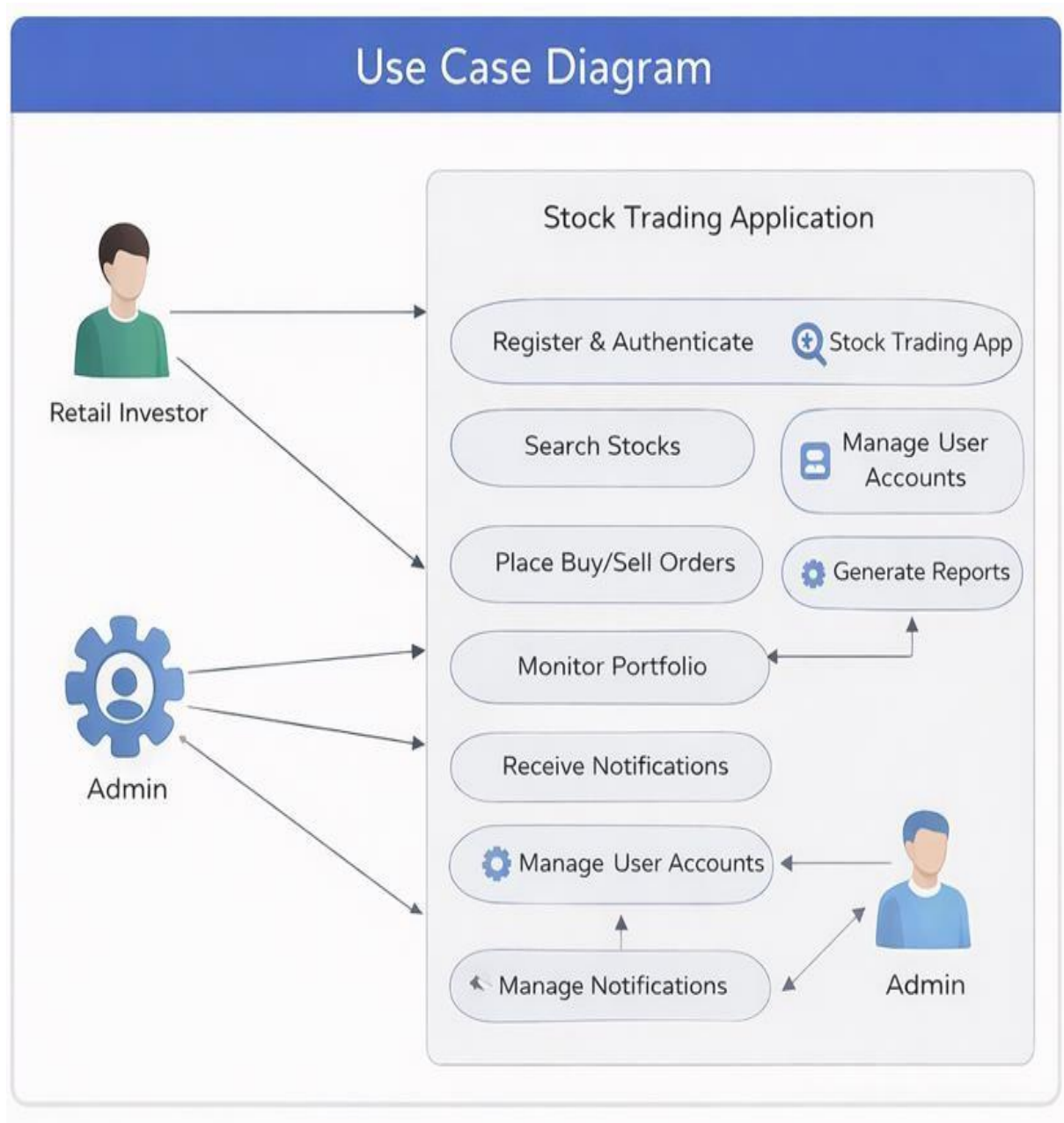
The system involves multiple stakeholders who contribute to the success of the project. Retail investors represent the primary users of the system, as they will use the application to manage their investments. The Jarvis organization serves as the project sponsor and provides the funding and strategic direction for the platform.

Trevor Doto acts as the project manager and client acceptor, overseeing project delivery and validating the final solution. Additional stakeholders include the development team responsible for building the system, the quality assurance team responsible for testing system behavior, and the DevOps team responsible for infrastructure and deployment.

Security and compliance teams also play a critical role by ensuring that the system adheres to financial data protection regulations such as PIPEDA.

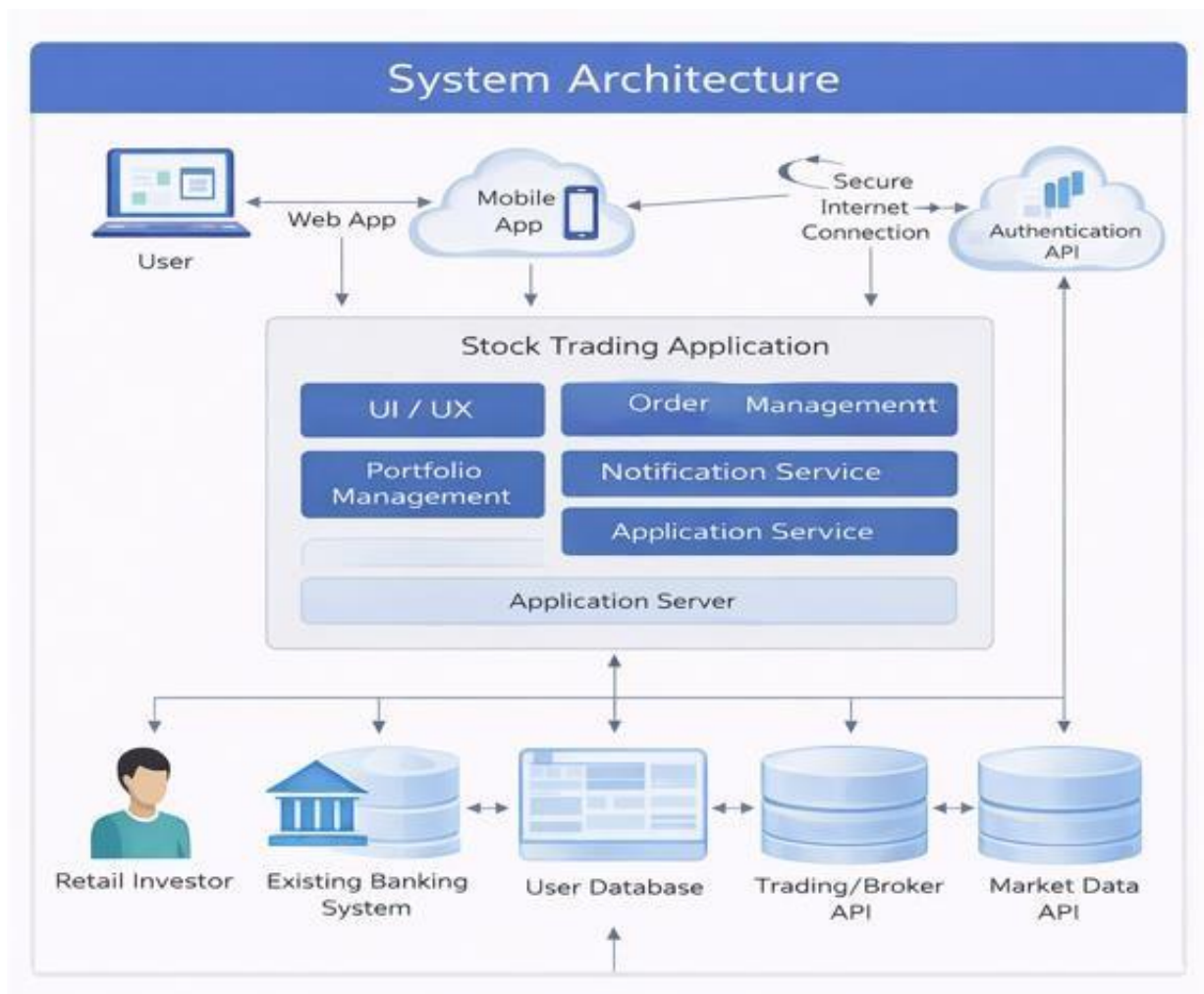
6. Use Case Diagram

This diagram illustrates the interactions between the primary users of the system and the core functionalities of the Stock Trading Application. Retail investors interact with the platform to search stocks, place buy or sell orders, monitor portfolios, and receive notifications. Administrative users manage system configurations, user accounts, and reporting features.



7. System Architecture

The system architecture diagram illustrates the high-level technical components of the Retail Banking Stock Trading Application. Users access the platform through web or mobile applications connected through a secure internet connection. The core trading system consists of multiple services including UI/UX, order management, portfolio management, notification services, and API integration layers. The application communicates with external systems such as the existing banking system, user database, trading broker APIs, and market data providers.



The platform follows a **microservices architecture** consisting of several independent services:

API Gateway

Handles incoming requests from web and mobile applications.

Example endpoints:

POST /api/orders

GET /api/portfolio

GET /api/stocks/{symbol}

POST /api/auth/login

Order Management Service

Responsible for validating and processing trade orders.

Data Flow:

1. Client submits order request
2. API Gateway forwards request to Order Service
3. Order Service validates funds and order parameters

4. Order sent to Broker API
5. Execution response returned to application

Portfolio Service

Calculates real-time portfolio valuation using market price feeds.

Market Data Service

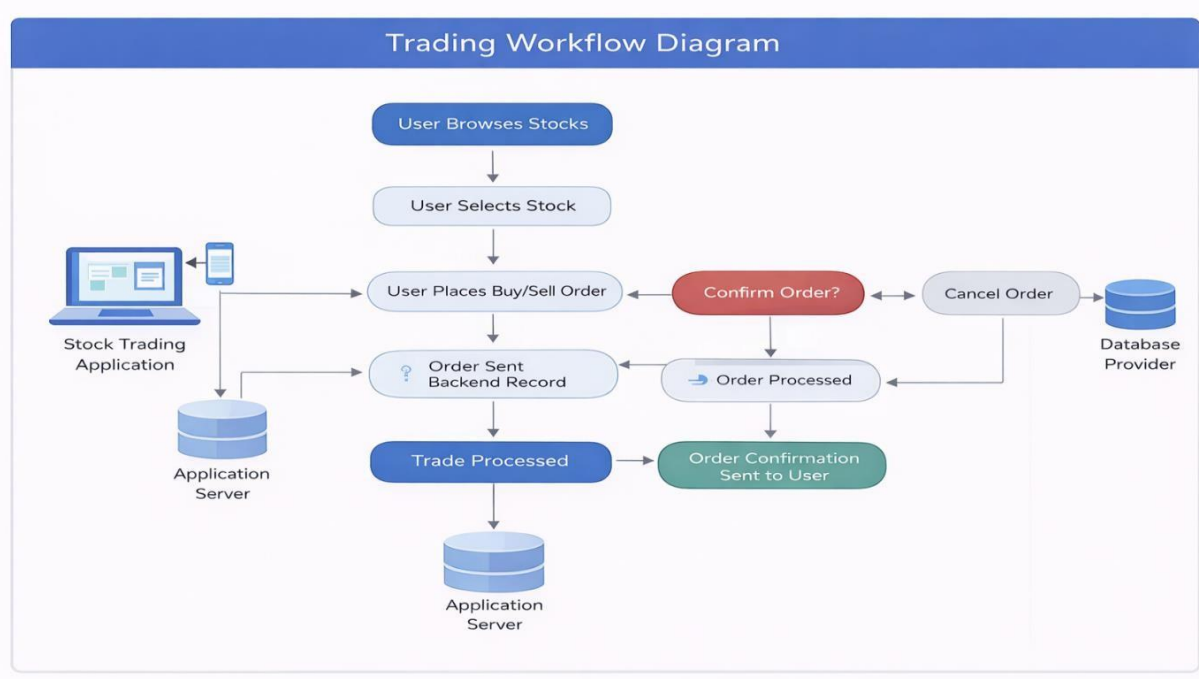
Retrieves stock price data from external market APIs.

Example provider:

- AlphaVantage
- IEX Cloud
- Bloomberg API

8. Trading Workflow

The trading workflow diagram represents the step-by-step process followed when a user executes a trade through the platform. The workflow begins when a user searches for stocks and selects a specific security. The user then places a buy or sell order which is sent to the backend trading system. After order confirmation, the request is processed through broker APIs, recorded in the backend database, and the system updates the user's portfolio. Finally, the user receives an order confirmation notification.

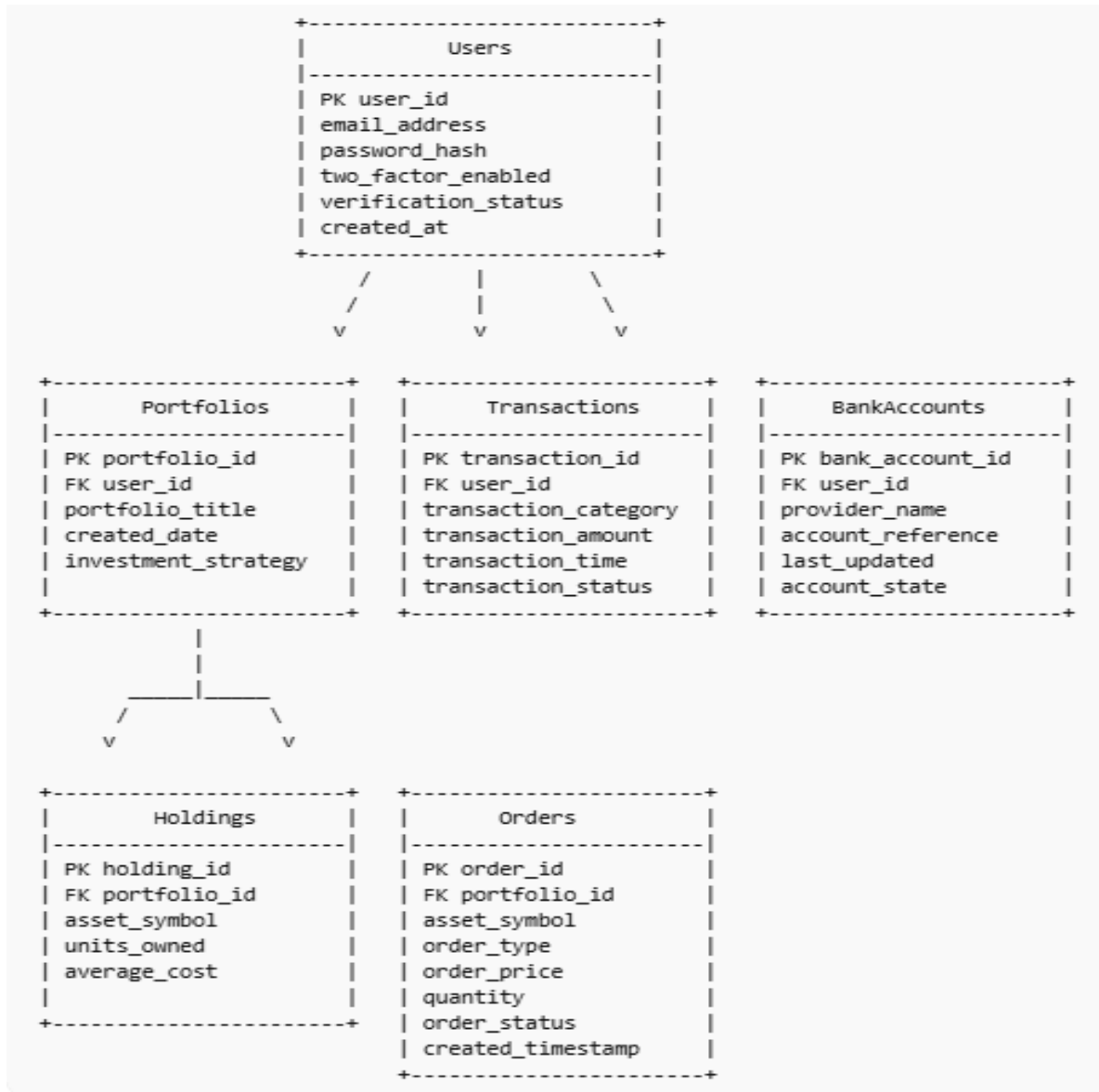


9. Data Model (ER Diagram)

The Entity Relationship (ER) diagram illustrates the logical database structure supporting the stock trading platform. The primary entity in the system is **Users**, which stores information related to registered customers of the platform. Each user can manage multiple **Portfolios**, link external **Bank Accounts**, and perform financial **Transactions** within the system.

A **Portfolio** represents a collection of investments owned by a user and serves as the parent entity for **Holdings** and **Orders**. The **Holdings** entity stores information about the assets currently owned in a portfolio, including the asset symbol, quantity, and average purchase cost. The **Orders** entity records trading activities such as buy or sell requests, including details such as order type, quantity, price, and order status.

The **Transactions** entity maintains records of financial activities related to the user account, such as deposits, withdrawals, or other account movements, while **Bank Accounts** represent external financial accounts linked to the platform for funding or settlement purposes. These relationships allow the system to maintain accurate records of user identities, portfolio assets, and trading activities while ensuring data integrity and traceability across the platform.



Data Model Constraints

Users Table

Field	Type	Constraint
user_id	UUID	Primary Key
email_address	VARCHAR(255)	Unique, Not Null
password_hash	VARCHAR	Not Null
two_factor_enabled	BOOLEAN	Default false

Orders Table

Field	Type	Constraint
order_id	UUID	Primary Key
portfolio_id	UUID	Foreign Key
asset_symbol	VARCHAR(10)	Not Null
order_type	ENUM	Market / Limit / Stop
order_price	DECIMAL	Must be >0
quantity	INTEGER	Must be >0
order_status	ENUM	Pending / Executed / Failed

10. Functional Requirements

FR-1 User Authentication

- The system shall authenticate users using the bank's identity provider API.
- The system shall support multi-factor authentication (MFA) using SMS or authenticator apps.
- After 5 failed login attempts, the system shall temporarily lock the account for 15 minutes.

FR-2 Stock Search

- The system shall allow users to search stocks using ticker symbols or company names.
- The system shall retrieve stock data from external Market Data APIs.

FR-3 Order Placement

The system shall support the following order types:

- Market Orders (executed immediately at current market price)
- Limit Orders (executed only when market price reaches the specified price)
- Stop Orders (executed when the stock reaches a trigger price)

Order validation rules:

- Order quantity must be greater than 0.
- Users must have sufficient account balance before submitting buy orders.
- Users must own sufficient shares before placing sell orders.

FR-4 Order Processing

- The system shall send validated orders to broker APIs.
- The system shall receive order execution confirmations and update order status.

Possible order states:

- Pending
- Submitted
- Executed
- Failed
- Cancelled

FR-5 Error Handling

The system shall detect and log the following failure scenarios:

- API connection failure
- Order rejection by broker
- Insufficient funds
- Invalid stock symbol
- Network timeout

The system shall display user-friendly error messages and store failure logs for audit purposes. The platform will also support several additional user functionalities that enhance the overall trading experience. The system will continuously calculate and display portfolio valuation by combining historical trade records with the latest available market price data retrieved from external market data providers. Users will be able to create and manage personalized watchlists to monitor stocks of interest and track price movements without executing trades. The application will provide real-time notification capabilities through channels such as email, SMS, and mobile push notifications to inform users about order confirmations, significant price changes, and important account activity. Additionally, users will have the ability to manage and customize their notification preferences, allowing them to enable or disable specific types of alerts according to their individual trading and monitoring needs.

11. Data Requirements

The system must store several types of financial and operational data to support trading functionality.

User accounts will be referenced through a unique identifier associated with their existing banking profile. Instead of duplicating personal information within the trading application,

the system will retrieve necessary user data from the bank's core systems.

Trade records must include information such as stock symbol, trade date, number of shares, and execution price. These records will be used to track portfolio performance and calculate investment gains or losses.

The system will also maintain information related to stock quotes and historical trade activity to enable users to review their investment history and portfolio performance.

12. Non-Functional Requirements

The trading platform must meet several non-functional requirements related to performance, scalability, security, and availability.

The application must support up to one million registered users while maintaining the ability to handle approximately two hundred thousand concurrent users during peak trading periods.

High system availability is critical because users must be able to access the application during market hours. The system must maintain approximately 99.97% uptime to ensure that users can execute trades during important market events.

Performance Metrics

The system performance will be evaluated using measurable indicators:

Metric	Target
System uptime	99.97% availability
API response time	< 300 ms
Order execution latency	< 2 seconds
Failed order rate	< 0.5%
Portfolio calculation time	< 1 second

Business Impact Metrics

- Reduction in broker-assisted trades by **40%**
- Increase in digital trading adoption by **25%**
- Reduction in order processing time by **60%**

Security and Compliance Requirements

Security requirements include encryption of all sensitive financial data, compliance with industry security standards, and adherence to privacy regulations such as PIPEDA. The

system must also maintain an audit trail of trading activity to ensure transparency and regulatory compliance.

The platform must implement the following security mechanisms:

Authentication Security

- OAuth 2.0 authentication for user sessions
- Multi-factor authentication for sensitive transactions

Data Protection

- AES-256 encryption for stored financial data
- TLS 1.3 encryption for all API communication

Access Control

- Role-based access control (RBAC) for admin and system users
- Principle of least privilege for system services

Compliance

The system must comply with regulations:

- **PIPEDA** for Canadian privacy protection
- **FINTRAC reporting requirements**
- **PCI DSS standards** for payment data protection

AuditLogging

All trading activities must generate immutable audit logs including:

- user_id
- order_id
- timestamp
- transaction result
- IP address

13. Infrastructure and Monitoring

The application will be hosted on cloud infrastructure, likely using Amazon Web Services (AWS) or similar enterprise cloud services. Monitoring and observability tools such as Prometheus, Grafana, and Splunk may be used to track system performance, uptime, and user activity. These tools will enable the operations team to monitor application health, detect system failures, and maintain service reliability.

14. Future Enhancements

While the initial release of the platform will focus on core stock trading functionality, the architecture will be designed to support additional financial instruments in the future. Potential enhancements include options trading, short selling, cryptocurrency trading, and foreign exchange investments.

By designing the system with extensibility in mind, it is possible to expand the capabilities of the platform without major architectural changes.